

Track B: Point Cloud & Mesh Reconstruction

Team: 1 human + 1 AI agent **Owns:** `/track_b_mesh` **Starts Day 1** against Track A's mock `depth.npy / intrinsics.json` in `/shared/sample_jobs/job_0001` — do not wait for Track A's real model.

1. What this track delivers

For any `depth.npy` + `intrinsics.json` pair: - `pointcloud.ply` — back-projected 3D points with color, arbitrary units - `mesh.obj` (or `.ply`) — a single triangulated mesh via Poisson surface reconstruction, minimal outlier cleanup, downsampled for speed (this is the MVP version — full tuned cleanup and Ball Pivoting are explicitly deferred past MVP week 1)

Exact shapes are fixed in `00_shared_contract.md` §2.

2. Who does what, and why

What the agent does: - Writes the depth-to-point-cloud back-projection code (using the pinhole intrinsics from Track A's `intrinsics.json`). - Writes the Poisson reconstruction pipeline (Open3D or equivalent): downsampling, minimal outlier removal, meshing. - Explains the *tradeoffs* live as they come up — e.g. “downsampling this much will speed things up but here’s what detail you’ll lose” — so the human is making an informed call, not just running a black box. - Tunes parameters (voxel size, Poisson depth) based on what the human reports seeing.

What the human does: - Opens the resulting `.ply / .obj` in an actual 3D viewer (MeshLab, Open3D viewer, Blender — whatever’s installed) and looks at it from multiple angles. This is not automatable in the MVP timeframe: “does this look like the object” is a visual judgment call. - Reports specific defects back in concrete terms the agent can act on: not “it looks bad” but “there’s a hole where the depth map had a gap” or “the back of the object is smeared into a cone” — the more specific the human’s report, the better the agent’s next fix. - Decides when quality is “good enough for MVP” vs. worth another iteration — this is a scope/time tradeoff call, and it’s the human’s call to make, not the agent’s to assume. - Runs the pipeline locally to get real timing numbers (mesh reconstruction can be slow; only the human’s machine will tell you the real number).

Why this split: the agent can write correct reconstruction code far faster than the human can, but has no way to *see* the output — it only knows the file was written successfully, not whether the mesh looks like the input object. The human supplies the eyes; the agent supplies the speed and the parameter-tuning knowledge.

3. Day-by-day

Day	Task
1	Agent scaffolds back-projection + Poisson pipeline against the mock <code>depth.npy</code> . Human runs it, confirms a <code>.ply / .obj</code> is produced and opens without errors.
2	Human opens output in a 3D viewer for the first time, reports what they see; agent adjusts point-cloud density / outlier removal accordingly.
3	Swap in Track A's <i>real</i> depth output (once

	available) for one hero image; human re-checks visually; agent tunes Poisson depth parameter for speed vs. quality.
4	Run all hero images through; human flags any that fail or look clearly wrong; write “what we built vs. contract” note.
5	Integration day — hand real mesh.obj / pointcloud.ply to Track C.

4. Definition of done

- pointcloud.ply and mesh.obj produced for all hero images, matching the contract format exactly (arbitrary units — do not scale here).
- Human has visually inspected every hero-image mesh at least once.
- One paragraph noting any known defects, so Track D can decide whether to exclude a hero image from the accuracy report rather than report a misleadingly bad number.

5. Explicitly out of scope for this week

Ball Pivoting as an alternative to Poisson, per-image outlier-parameter tuning, and higher point density are all real accuracy improvements — they are the first three items on the post-MVP upgrade backlog, not part of this week’s deliverable. Don’t scope-creep into them; flag them for later instead.